

CS-AML

Technical Stack and Repository Specification

Version 0.1.4

Approved Internal Specification Baseline

2026-10-10 · tag v0.1.4-spec

Civil Society Anti-Money Laundering & Financial Intelligence Framework

Generated from the canonical Markdown source. The Markdown file in the repository remains authoritative; change tags such as `[v0.1.x · ...]` are part of the traceability record.

Contents

1. Purpose and Relationship to the CS-AML Suite	4
2. Engineering Principles	5
3. MVP Reference Stack	5
4. Architecture Style: Modular Monolith	7
5. Repository Strategy	7
6. Backend Repository Structure	8
7. Frontend Repository Structure	8
8. Canonical Data Storage Rules	9
9. Evidence Storage and Integrity	9
10. Identity and Authorization Implementation	10
11. API Design Standard	11
12. Background Jobs and Idempotency	12
13. Search Strategy for MVP	12
14. Graph Strategy for MVP	12
15. Product Generation and Document Output	13
16. Configuration and Secrets	13
17. Environment Model	13
18. Container and Runtime Topology	13
19. Docker Image and Supply-Chain Rules	14
20. CI/CD Pipeline	14
21. Testing Stack and Test Pyramid	15
22. Code Quality and Conventions	15
23. Database Migration Policy	15
24. Audit Implementation Standard	16
25. Observability and Logging	16
26. Backup, Restore and Disaster Recovery	16
27. External Integration Boundary	16
28. Security Hardening Baseline	17
29. Performance and Capacity Baseline	17
30. Accessibility and Analyst UX Engineering Rules	18
31. Feature Flags	18

32. Architecture Decision Records	18
33. Repository Documentation Set	18
34. MVP Module-to-Epic Mapping	19
35. Initial Delivery Sequence	19
36. Technical Definition of Done	19
37. Technical Release Gate for MVP 0.1	20
38. Deferred Technical Decisions	20
39. Conformance Checklist	20
40. Reference Repository Tree (Expanded)	21
41. Closing Engineering Decision	22

CS-AML

Technical Stack & Repository Specification

Version 0.1.4

Document status — v0.1.4

Version: 0.1.4 — Approved Internal Specification Baseline (2026-10-10, tag v0.1.4-spec). Supersedes v0.1.3 (2026-10-09, tag v0.1.3-spec). [v0.1.4]

Supersedes: CS-AML Technical Stack & Repository Specification v0.1. The DOCX/PDF files in this repository are the unchanged v0.1 baseline (legacy); this Markdown file is the canonical source.

Validation: approved by the product owner as the internal specification baseline on 2026-10-08 (v0.1.1) and 2026-10-09 (v0.1.2 and v0.1.3) and 2026-10-10 (v0.1.4; decision register and release gates in CHANGELOG.md). The v0.1.2, v0.1.3 and v0.1.4 changes come from change requests raised while implementing increments I1-I4, I5-I7 and the post-MVP follow-ups; this is not an independent audit. Acceptance criteria in this document are targets, not evidence that tests have passed.

CS-AML is not an external standard or certification. References to FATF, Wolfsberg, PPATK, UNODC or other bodies do not imply their endorsement.

Changes in 0.1.1: see CHANGELOG.md at the repository root (audit findings A01-A16).

Changes in 0.1.2: change requests CR-I1-01...CR-I4-14 approved by the product owner on 2026-10-09 (CHANGELOG.md, section v0.1.2). Each change is tagged *[v0.1.2 · CR-xx-yy]*.

Changes in 0.1.3: change requests CR-I5-01...CR-I7-08 approved by the product owner on 2026-10-09 (CHANGELOG.md, section v0.1.3). Each change is tagged *[v0.1.3 · CR-xx-yy]*.

Changes in 0.1.4: change requests CR-N-01...CR-N-14 approved by the product owner on 2026-10-10 (CHANGELOG.md, section v0.1.4). Each change is tagged *[v0.1.4 · CR-N-xx]*.

Engineering implementation baseline for the CS-AML MVP

Status: Proposed normative implementation specification (draft for review) [v0.1.1 · A01]

Architecture style: Modular monolith with replaceable infrastructure services

Target: CS-AML MVP 0.1

Core engineering axiom

Keep the domain model canonical, auditable, and portable. Infrastructure accelerators such as search, graph projections, caches, AI, and external connectors MUST remain replaceable and MUST NOT become the sole source of analytical truth.

1. Purpose and Relationship to the CS-AML Suite

This specification translates the CS-AML Technology Architecture, Software Requirements Specification (SRS), and MVP Engineering Breakdown into concrete technology and repository decisions for MVP 0.1. It defines the baseline stack, code boundaries, development conventions, runtime dependencies, repository layout, CI/CD expectations, environment model, and technical decision controls.

The document is intentionally more prescriptive than the Technology Architecture. Technology choices in this document are implementation decisions for the reference MVP and MAY be replaced through an Architecture Decision Record (ADR) when equivalent CS-AML requirements remain satisfied.

Upstream document	This specification consumes
Technology Architecture v0.1.1	Logical boundaries, canonical vs derived data, security and deployment principles
Data Model Specification v0.1.4	Canonical objects, identifiers, provenance, temporal semantics and integrity rules

Upstream document	This specification consumes
PRD v0.1.1	MVP product scope and release objective
SRS v0.1.4	Testable software requirements and release gate
MVP Engineering Breakdown v0.1.1	Epics, vertical slices, stories and delivery sequence
API Specification v0.1.4	HTTP contract, authentication (BFF session), concurrency and idempotency rules [v0.1.1 · A01, A11]

2. Engineering Principles

ID	Principle	Normative decision
TS-PR-01	Modular monolith first	MVP SHALL be implemented as a modular monolith for domain/application logic. Independent infrastructure processes MAY be separate containers.
TS-PR-02	Canonical truth in PostgreSQL	Canonical analytical records SHALL use transactional relational storage. Search indexes, graph projections, vector indexes and caches are rebuildable.
TS-PR-03	Evidence outside relational blobs	Original evidence SHALL be stored in an evidence object store with immutable version semantics; metadata and hashes remain canonical in PostgreSQL.
TS-PR-04	Policy at the API boundary	Authorization SHALL be enforced server-side for every protected operation. UI hiding is never a security control.
TS-PR-05	Async only where useful	Long-running ingestion, hashing, exports and projection jobs SHOULD be asynchronous; ordinary domain commands SHOULD remain synchronous where transactional correctness benefits.
TS-PR-06	Traceability over magic	Automated output MUST preserve source, rule/model version, time and canonical references.
TS-PR-07	Portable deployment	The application SHALL run on a single Linux host using containers and SHALL not require Kubernetes for MVP.
TS-PR-08	Extract later, not early	A module becomes a standalone service only when scaling, security isolation, ownership or operational evidence justifies extraction.

3. MVP Reference Stack

The following stack is the proposed CS-AML MVP reference profile (draft for review). [v0.1.1 · A01] Exact patch versions SHOULD be pinned in lockfiles and container digests. Major-version changes require compatibility testing and, where they affect architecture or security assumptions, an ADR.

Layer	Baseline	Rationale	MVP status
Operating system	Ubuntu Server 24.04 LTS	Stable deployment baseline; strong package/security ecosystem	Required reference
Containers	Docker Engine + Compose v2	Simple reproducible deployment for small teams	Required reference
Backend runtime	Python 3.12	Long support horizon; mature security/data ecosystem	Required reference
Web framework	Django 5.2 LTS	ORM, migrations, admin primitives, mature auth/ecosystem	Required reference
API	Django REST Framework + OpenAPI 3.1 schema committed as contracts/openapi.yaml	Explicit REST contract and testability	Required reference [v0.1.1 · A11]
Database	PostgreSQL 17	Canonical transactional store; JSONB/FTS available	Required reference
GIS extension	PostGIS 3.5	Optional geographic capability without separate GIS store	Enabled where GIS used
Async jobs	Celery 5.x	Evidence processing, export and projection jobs	Required reference

Layer	Baseline	Rationale	MVP status
Broker/cache	Valkey 8.x (BSD-3-Clause), Redis-protocol compatible; Celery uses the redis:// transport scheme. Exact minor/patch release pinned in the dependency manifest/ image digest; licence recorded in the SBOM (ADR-0006)	Task broker, bounded cache, transient coordination	Required reference [v0.1.1 · A03]
Object storage	S3-compatible object store with versioning/ object-lock-capable features; concrete product selected per ADR-0005 (MinIO is no longer the reference)	Original/derivative evidence objects, versioning	Required capability [v0.1.1 · A02]
Identity provider	Keycloak 26.x / standards-compliant OIDC; Django is a confidential OIDC client (Authorization Code + PKCE) and the browser holds only a server-side session cookie (BFF)	Central identity, MFA and session policy	External dependency [v0.1.1 · A11]
Frontend	React 19 + TypeScript + Vite	Typed analyst UI and component ecosystem	Required reference
CSS/UI	Tailwind CSS 4 + accessible component primitives	Consistent UI without proprietary design system	Reference
Graph UI	Cytoscape.js	Evidence-backed network exploration in browser	MVP visualization
Search	PostgreSQL FTS + trigram indexes	Avoid premature search cluster; permission-aware MVP search	MVP
OCR / text extraction	Tesseract 5.5 (Apache-2.0) with ind + eng tessdata 4.1; pdfium via pypdfium2 for PDF text layers and rendering; no OCRmyPDF / Ghostscript (AGPL)	Offline, permissive licences, pinned packages; traineddata SHA-256 recorded with every result	Post-MVP (v0.1.4) [v0.1.4 · CR-N-09]
Machine translation	CTranslate2 4.x (MIT) running OPUS-MT id ↔ en models (CC-BY 4.0) baked into the image with pinned SHA-256, re-verified before use; sacremoses + subword-nmt tokenisation	Offline only; no network path, no online fallback	Post-MVP (v0.1.4) [v0.1.4 · CR-N-10]
Graph persistence	Relational canonical model	No graph DB dependency for MVP	MVP
Reverse proxy	Nginx	TLS termination, headers, request controls	Reference
App server	Gunicorn	Django WSGI/ASGI serving baseline	Reference
Observability	OpenTelemetry + Prometheus/Grafana compatible metrics	Vendor-neutral instrumentation	Required capability
CI/CD	GitHub Actions or equivalent	Automated quality/security/ build gates	Required capability

Version policy

The versions above are baseline major/minor families, not an instruction to remain on insecure patches. Security patches SHOULD be applied within supported lines after automated and regression testing. Dependency updates SHALL NOT silently alter analytical semantics.

Dependency licence and maintenance note [v0.1.1 · A02, A03]

A version family does not imply a single licence. For each infrastructure dependency the pinned release, its licence and its maintenance status SHALL be recorded in the dependency manifest, the image digest and the SBOM, and they SHALL refer to the same version. Do not describe a release line as open-source or BSD without checking its licence: for example Redis ≤7.2 is BSD-3-Clause, Redis 7.4 is RSALv2/SSPLv1, and Redis 8.x adds AGPLv3 — which is why the broker/cache baseline is Valkey 8.x (ADR-0006). The upstream MinIO Community repository (minio/minio) was archived on 25 April 2026 and states it is no longer maintained; MinIO Community and commercial products (e.g. AIStor) are different products with different licences and support paths. The object store product is therefore chosen through ADR-0005, which SHALL record the chosen product and release line, maintenance status, licence, patch path, S3 compatibility test evidence, and backup/restore test evidence.

4. Architecture Style: Modular Monolith

CS-AML MVP SHALL use one deployable backend application containing strongly separated domain modules. This is a deliberate engineering choice: case workflow, evidence provenance, entity resolution, value-flow reasoning, review and dissemination frequently require consistent authorization, transaction boundaries and audit events. Premature service decomposition would add distributed consistency and authorization complexity before the product model stabilizes.

```

Browser (React)
|
v
Nginx / API boundary
|
v
Django modular monolith
|-- cases
|-- evidence
|-- entities / relationships / assets
|-- timeline / valueflows
|-- typologies / hypotheses / assessments
|-- search / graph projection API
|-- products / reviews / dissemination
|-- policy / audit / administration
|
+--> PostgreSQL (canonical)
+--> object-store (S3-compatible; product per ADR-0005)
+--> Valkey / Celery workers
+--> Keycloak (OIDC; server-side BFF session)
+--> rebuildable search/graph projections

```

[v0.1.1 · A02, A03, A11] — diagram updated: object store per ADR-0005; Valkey replaces Redis; browser authentication is a server-side BFF session.

Microservice extraction MAY occur after MVP when one or more of the following is demonstrated: materially different scaling profile; security isolation requirement; independent deployment cadence; dedicated operational ownership; external reuse; or unacceptable coupling that cannot be solved through module boundaries.

5. Repository Strategy

The MVP SHALL use a monorepo. Product documentation, API code, frontend, infrastructure definitions and integration tests remain version-aligned. This supports deterministic traceability from SRS requirement to implementation and test.

```

cs-aml/
├─ README.md
├─ LICENSE
├─ CONTRIBUTING.md
├─ SECURITY.md
├─ CHANGelog.md
├─ pyproject.toml
├─ package.json
├─ compose.yaml
├─ .env.example
├─ .editorconfig
├─ .github/
│   └─ workflows/
│       └─ CODEOWNERS
├─ backend/
├─ frontend/
├─ contracts/      # openapi.yaml (OpenAPI 3.1, generated, linted in CI)
├─ infra/
├─ docs/
├─ scripts/
└─ tests/

```

| └─ fixtures/
[v0.1.1 · A11] — contracts/openapi.yaml added (see API Specification v0.1.4 §28).

6. Backend Repository Structure

```
backend/  
├─ manage.py  
├─ config/  
│   ├── settings/  
│   │   ├── base.py  
│   │   ├── local.py  
│   │   ├── test.py  
│   │   └─ production.py  
│   ├── urls.py  
│   ├── asgi.py  
│   └─ celery.py  
├─ csaml/  
│   ├── common/  
│   ├── iam/  
│   ├── cases/  
│   ├── sources/  
│   ├── evidence/  
│   ├── entities/  
│   ├── relationships/  
│   ├── assets/  
│   ├── timeline/  
│   ├── valueflows/  
│   ├── typologies/  
│   ├── hypotheses/  
│   ├── assessments/  
│   ├── search/  
│   ├── graph/  
│   ├── products/  
│   ├── reviews/  
│   ├── dissemination/  
│   ├── governance/  
│   ├── audit/  
│   └─ integrations/  
└─ tests/
```

Each domain package SHOULD expose explicit application services and policy checks. Views/serializers SHALL NOT contain material domain logic. Direct model access across modules SHOULD be minimized; cross-module operations SHOULD call a documented application service or selector.

Module element	Responsibility
models.py / models/	Persistent canonical model and invariant-adjacent constraints
services.py / services/	State-changing use cases and transaction boundaries
selectors.py	Read/query functions with authorization-aware filters
policies.py	Object/action authorization and need-to-know policy
api/	Serializers, request validation, routes and response mapping
tasks.py	Idempotent asynchronous work; never hidden authorization bypass
events.py	Explicit domain/application events used for audit/projections
tests/	Unit, policy, service, API and regression tests

7. Frontend Repository Structure

| frontend/

```

|— src/
|   |— app/
|   |— routes/
|   |— features/
|   |   |— cases/
|   |   |— evidence/
|   |   |— entities/
|   |   |— investigation-graph/
|   |   |— timeline/
|   |   |— value-flows/
|   |   |— typologies/
|   |   |— hypotheses/
|   |   |— assessments/
|   |   |— products/
|   |   |— reviews/
|   |— components/
|   |— api/
|   |— auth/
|   |— design-system/
|   |— test/
|— e2e/

```

Frontend features SHALL mirror product capabilities rather than backend table names. The browser SHALL NOT be responsible for deciding whether an operation is authorized, whether a relationship is valid, or whether an analytical class can transition; it may only present server-provided permissions and validation outcomes.

8. Canonical Data Storage Rules

ID	Requirement
DB-01	PostgreSQL is canonical for analytical records, metadata, workflow state, access relationships and audit references.
DB-02	Evidence binaries SHALL NOT be stored as large database blobs unless a deployment ADR explicitly chooses that mode.
DB-03	Every canonical object SHALL use stable UUID identifiers and created/updated/version metadata.
DB-04	Material deletes SHOULD be logical/superseding operations where historical reconstruction is required.
DB-05	Entity merge/unmerge SHALL preserve merge decision history and original identifiers. Decisions are stored in an append-only resolution_decisions table (Data Model v0.1.4 §8.4); entity resolution_status is updated only as their effect. [v0.1.1 · ER]
DB-06	ValueFlow.flow_class SHALL be constrained to the UPPER_SNAKE_CASE values DIRECT, DOCUMENTED, RECONSTRUCTED, HYPOTHETICAL; confidence.level to HIGH, MODERATE, LOW, INSUFFICIENT_BASIS (never coerced to LOW/null/zero); classification to PUBLIC, INTERNAL, SENSITIVE, RESTRICTED, SOURCE_PROTECTED. Enum values derive from the Data Model Annex A registry. [v0.1.1 · A08, A09]
DB-07	Unknown numeric values SHALL remain null/unknown and SHALL NOT be coerced to zero.
DB-08	Canonical records SHALL expose provenance links sufficient to reconstruct material assessments.

9. Evidence Storage and Integrity

Evidence storage consists of canonical metadata in PostgreSQL plus immutable/versioned objects in S3-compatible storage (versioning/object-lock-capable; product per ADR-0005). [v0.1.1 · A02] User-visible filenames are metadata only; object keys SHALL be opaque and collision-resistant.

Concern	MVP requirement
Original preservation	Original object cannot be overwritten in place by analyst workflows.

Concern	MVP requirement
Hashing	SHA-256 recorded at or immediately after acquisition for critical evidence.
Derivatives	OCR, translation, crop, parsed table or redacted copy gets a new object and parent lineage. OCR / extraction / translation output is a canonical JSON document stored as a derivative EvidenceItem plus a DerivedText record (Data Model §7.7); the worker re-hashes the pinned original before processing (mismatch → FAILED). <i>[v0.1.4 · CR-N-09, CR-N-10]</i>
Automation engines	Every automation engine (OCR, machine translation, any future AI) runs offline in the worker; engine modules import no network library, models are pinned and integrity-checked, and evidence content never leaves the host. Switches: CSAML_OCR_ENABLED, CSAML_OCR_AUTO_AFTER_INGEST (default off), CSAML_MT_ENABLED, CSAML_DERIVED_TEXT_SEARCH. <i>[v0.1.4 · CR-N-10]</i>
Download authorization	Every download is re-authorized server-side; direct bucket browsing is not exposed.
Malware handling	Uploads SHOULD be scanned/quarantined before routine analyst download where deployment risk warrants it.
Metadata	MIME type, byte size, original filename, object key, hash, acquisition time and collector retained.
Backup	Evidence objects included in backup/replication policy independently from DB metadata.

10. Identity and Authorization Implementation

Keycloak is the reference identity provider, but CS-AML SHALL depend on OIDC semantics rather than Keycloak-specific session logic. Local authorization remains in CS-AML because case membership, classification, protected-source access, object state and independent-review constraints are application-domain rules.

```

Identity decision: IdP / OIDC
    |
    v
Local user + organisation mapping
    |
    v
CS-AML authorization policy
(role + case membership + object classification + action + workflow state)
    |
    v
Allow / deny + audit context

```

- Browser authentication SHALL use a server-side session (BFF): Django is a confidential OIDC client of Keycloak (Authorization Code + PKCE); access/refresh/ID tokens stay server-side; the browser holds only the `__Host-csaml_session` cookie (HttpOnly, Secure, SameSite=Lax, Path=/) and sends X-CSRFToken on unsafe methods. Endpoints `/auth/login`, `/auth/callback`, `/auth/logout`, `/auth/session` are defined in the API Specification v0.1.4 §4. *[v0.1.1 · A11]* The CSRF token is delivered as `csrf_token` in GET `/auth/session`; its secret lives in the server-side session and there is no JavaScript-readable CSRF cookie. Logout is a top-level form POST that may carry the token as `csrfmiddlewaretoken`. POST `/auth/backchannel-logout` (OIDC Back-Channel Logout 1.0, signed `logout_token`, CSRF-exempt) lets Keycloak revoke server-side sessions. *[v0.1.2 · CR-I1-03, CR-I1-04, CR-I1-05]*
- Principal attribute model (clearance). Object classification is matched against IdP realm roles carried in the session: without a clearance role a principal is cleared to INTERNAL (and PUBLIC); `csaml-clearance-sensitive` clears up to SENSITIVE; `csaml-clearance-restricted` clears up to RESTRICTED. SOURCE_PROTECTED access needs the eligibility role `csaml-protected-source` plus the explicit per-case membership grant `protected_source_authorized` (neither alone suffices). Access labels are matched by roles `csaml-label-<label>`; a principal needs a role for every label on an object. Unknown or missing classifications or labels fail closed (deny). Clearance is a ceiling on top of case membership and role checks, never a substitute for them (Data Model §16). *[v0.1.2 · CR-I1-10]*

- MFA policy SHOULD be enforced by the IdP and verified through authentication context/claims where available.
- Step-up for high-risk actions: Keycloak levels of authentication ACR 1 = password, ACR 2 = password + TOTP (browser flow with Level-of-Authentication conditions; level 2 max age 0, so every ACR-2 request asks for the code; TOTP SHA-1, 6 digits, 30 s, codes not reusable; users without OTP enrol at their first step-up). The BFF sends `acr_values=2` for GET `/auth/login?acr=2`, records the verified `acr` and `auth_time`, and enforces `ACR ≥ 2` within `CSAML_STEP_UP_MAX_AGE_SECONDS` (default 900 s; production refuses < 60 s) for the action list in API Specification §4 (403 STEP_UP_REQUIRED). The realm configuration is applied idempotently by a script and mirrored in the development realm file. TOTP is not phishing-resistant; WebAuthn / passkeys for approvers and per-deployment action lists are deferred to v0.2. *[v0.1.4 · CR-N-13]*
- Administrative roles and investigative roles SHOULD be separable; platform administrators SHOULD NOT automatically receive access to case content.
- ProtectedSource identity SHALL use an explicit compartment and dedicated permission.
- Every export, disclosure, merge, gate approval and review decision SHALL execute a server-side authorization policy.

11. API Design Standard

The MVP API SHALL be REST/JSON and versioned under `/api/v1/`. The OpenAPI 3.1 contract `contracts/openapi.yaml` (v0.1.2: the P0 vertical slice plus the I3/I4 operations formerly kept as implementation extensions, merged into the single contract *[v0.1.2 · CR-I3-01, CR-I4-01]*; written contract-first) SHALL be linted and checked in CI for breaking changes, used to generate the TypeScript client, and exercised by contract tests; once code exists, the schema generated from the implementation SHALL be diffed against it. *[v0.1.1 · A11]* IDs SHALL be opaque UUIDs. Internal database primary-key assumptions SHALL NOT appear in public contracts.

Area	Convention
Base path	<code>/api/v1/</code>
Authentication	Server-side OIDC session (BFF); session cookie only, no bearer tokens from browsers; X-CSRFToken on every unsafe method; same-origin, CORS disabled by default <i>[v0.1.1 · A11]</i>
Errors	Stable machine code + human message + correlation ID; no sensitive existence leakage
Pagination	Cursor or stable page pagination for large collections
Filtering	Explicit allowlisted fields; permission filtering occurs before result shaping
Concurrency	ETag: " <code><record_version></code> " on GET; If-Match REQUIRED on mutations of versioned resources; missing → 428, stale → 412 PRECONDITION_FAILED; 409 STATE_CONFLICT for workflow conflicts only <i>[v0.1.1 · A04]</i> ; exception: multi-entity commands (merge, unmerge, resolution and match-candidate decisions) send body <code>expected_versions</code> instead of If-Match (missing → 428, mismatch → 412 with details <code>current_record_versions</code>) <i>[v0.1.1 · C03]</i>
Idempotency	Idempotency-Key (UUID) REQUIRED for evidence ingest finalization, merge/unmerge, review/dissemination approval, export generation; SHOULD for creates; scope, 24 h retention and replay/conflict semantics per API Specification §11 <i>[v0.1.1 · A11]</i>
Schema	Generated OpenAPI 3.1 in <code>contracts/openapi.yaml</code> ; lint and contract diff in CI <i>[v0.1.1 · A11]</i>

High-impact actions SHOULD use command-style endpoints when a generic CRUD update would obscure required checks, for example `/entities/merge`, `/reviews/{id}/approve`, `/disseminations/{id}/approve`, and `/cases/{id}/gates/{gate}/approve` (aligned with the API Specification v0.1.4 endpoint catalogue). *[v0.1.1 · A11]*

12. Background Jobs and Idempotency

Job class	Examples	Rule
Evidence processing	Hashing, metadata extraction, malware scan	Job result links to evidence version; retry-safe
Exports	Report package generation	Generate immutable product/export version; no silent overwrite
Derived text	OCR / text extraction, machine translation (jobs TEXT_EXTRACTION, MACHINE_TRANSLATION)	Offline engines; worker re-checks the requester's rights; per-page timeout and page limit; output immutable once complete [v0.1.4 · CR-N-09, CR-N-10, CR-N-12]
Projection	Search document or graph projection refresh	Rebuildable; canonical transaction commits first
Notifications	Review/task notifications	Never include sensitive case payload in external notification by default
Maintenance	Retention preview, cleanup, integrity verification	Dry-run and auditable for destructive operations

Celery tasks SHALL be idempotent or carry deduplication/idempotency keys. A failed projection task MUST NOT roll back a committed canonical analytical action.

13. Search Strategy for MVP

MVP search SHALL use PostgreSQL full-text search, trigram matching and purpose-built indexes. This avoids a second authorization/indexing system before search requirements stabilize. Search is permission-aware at query time and MUST NOT reveal counts, snippets or existence of inaccessible objects.

- Index case title/metadata, source metadata, evidence extracted text when permitted, entity names/aliases/identifiers, relationships, assets, events and products.
- MVP 0.1 index scope (decided): cases; entities (names, aliases, identifiers); sources; evidence **metadata**; extracts (cited text); claims; facts; relationships (type and dates only — never endpoint names); assets; events; value flows; products. Evidence file content (OCR / full text) is not indexed in MVP 0.1 (Phase 2); since v0.1.4 completed DerivedText is indexed as DERIVED_TEXT / DERIVED and can be switched off [v0.1.4 · CR-N-11]; hypotheses, assessments and indicators are not indexed yet. The projection (search_documents) is rebuildable from canonical records; authorization is applied in SQL before results, counts, facets, snippets and ordering, and relevance uses per-document scores without corpus statistics. Limits: search 120 and graph 60 requests per minute per user, search statement timeout 5 s (API §17, §18, §25). [v0.1.3 · CR-I5-01, CR-I5-07, CR-I5-08]
- Search results MUST link to canonical object IDs and display object type, case context and provenance cues.
- OpenSearch or equivalent is Phase 2 only when corpus size, faceting, highlighting or cross-language performance justifies it.
- A search index is always rebuildable and never authoritative for object state or access policy.

14. Graph Strategy for MVP

Canonical relationships remain relational first-class objects in PostgreSQL. The MVP investigation graph is a UI projection rendered with Cytoscape.js. The backend returns authorized nodes and relationship records with evidence/status/confidence metadata.

Graph rule

An edge shown on screen MUST correspond to an authorized canonical Relationship, OwnershipInterest, ControlAssertion or ValueFlow reference. Pure layout proximity or algorithmic suggestion MUST NOT be rendered as an established relationship.

Neo4j/Memgraph MAY be added as a derived projection in Phase 2 for path queries and network analytics. It SHALL be reconstructible from canonical records and SHALL NOT become the only copy of relationship provenance.

15. Product Generation and Document Output

Intelligence products SHALL be generated from versioned assessments and evidence references. Server-side templates SHOULD produce DOCX/PDF or another controlled export format. Export generation is a high-impact operation and SHALL record product version, included objects, handling classification, author/reviewer, approval and recipient context where applicable.

- Final product generation MUST NOT query unrestricted global data outside the authorized product scope.
- Evidence indexes SHALL resolve to canonical EvidenceItem/EvidenceExtract records.
- Corrections create a new product version and mark superseded versions; history remains accessible to authorized reviewers.
- Export packages SHOULD support redaction/minimization without mutating canonical source objects.

16. Configuration and Secrets

Class	Examples	Rule
Non-secret config	public base URL, feature flags, limits	Environment or config file; versionable defaults
Secret	DB password, OIDC client secret, Django session/CSRF secret key, object-store (S3-compatible) access key, Valkey password	Secret store/environment injection; never committed [v0.1.1 · A02, A03, A11]
Security policy	idle and absolute session timeout, export limits, protected-source policy	Central server config; changes audited where material
Controlled vocabulary	relationship types, classifications, statuses	Canonical DB with versioning and admin audit

.env.example SHALL contain names and safe examples only. Production secrets SHALL be provisioned outside the repository. Secret rotation procedures SHALL not require rebuilding application source code.

17. Environment Model

Environment	Purpose	Data rule
local	Developer workstation and feature work	Synthetic fixtures only
test/CI	Automated unit/integration/security tests	Ephemeral synthetic data
staging	Release candidate, migration and E2E validation	Synthetic or formally sanitized datasets only
production	Operational investigations	Real sensitive data under approved controls

Production data SHALL NOT be copied into developer or CI environments. Any diagnostic export from production requires minimization, authorization, and documented deletion.

18. Container and Runtime Topology

```
| compose project (reference deployment)
| └─ nginx
```

```

├─ web          (Django/Gunicorn)
├─ worker       (Celery)
├─ scheduler    (Celery beat, only if required)
├─ postgres
├─ valkey       (Valkey 8.x, pinned digest; ADR-0006)
├─ object-store (S3-compatible; product per ADR-0005; external S3-compatible service supported)
├─ telemetry    (deployment-specific collectors)

```

External or separately managed:

```

├─ Keycloak / OIDC IdP

```

[v0.1.1 · A02, A03] — MinIO is no longer the reference object store; Redis is replaced by Valkey.

Database and object storage **SHOULD** be separately backed up and **SHOULD** use persistent volumes not coupled to application container lifecycle. Production deployments **SHOULD** pin container image digests after build/release approval.

19. Docker Image and Supply-Chain Rules

- Use multi-stage builds and non-root runtime users where practical.
- Pin direct dependencies through lockfiles; container base images **SHALL** use supported minimal variants.
- CI **SHALL** run dependency vulnerability scanning and container image scanning before release.
- Generated SBOM **SHOULD** be attached to release artifacts and **SHALL** record the licence of each pinned infrastructure dependency (including the broker/cache and object store).
[v0.1.1 · A03]
- Production images **SHALL** be built in CI, not on the production server.
- Release tags **SHALL** map to immutable image digests and application build metadata.

20. CI/CD Pipeline

```

Pull Request
-> formatting/lint
-> unit tests
-> policy/authorization tests
-> migration checks
-> frontend tests
-> SAST/dependency scan
-> build images
-> integration tests
-> review/merge

Release Candidate
-> signed/tagged build
-> deploy staging
-> migrate + smoke test
-> end-to-end regression
-> backup/restore evidence where required
-> approval
-> production deploy

```

Gate	Minimum evidence
Code quality	Ruff/mypy or equivalent backend checks; ESLint/TypeScript frontend checks
Tests	Unit + integration + authorization tests pass
Migrations	No unapplied model drift; migration plan reviewed
Security	Dependency/SAST/image findings triaged; release blocker policy applied

Gate	Minimum evidence
Contract	contracts/openapi.yaml generation and lint succeed; contract tests pass; intentional breaking changes documented [v0.1.1 · A11]
Release	Version, changelog, build metadata and rollback/migration notes available

21. Testing Stack and Test Pyramid

Layer	Reference tools	Coverage target
Backend unit/service	pytest + pytest-django	Domain invariants, service logic, authorization policy
API integration	pytest + DRF test client	Request validation, permissions, audit, transactional effects
Frontend unit/component	Vitest + Testing Library	Critical forms, state, uncertainty labels, permission rendering
End-to-end	Playwright	MVP release path from case to approved product
Security regression	pytest/Playwright + scanning tools	Object access isolation, export gates, protected-source leakage
Migration tests	Django migration test harness	Forward migration, representative data evolution, rollback plan

Authorization and provenance tests are first-class product tests, not optional security extras. Every P0 high-impact operation SHOULD have at least one negative authorization test.

22. Code Quality and Conventions

Area	Baseline
Python formatting/lint	Ruff; deterministic formatting/lint rules
Python typing	mypy or equivalent on service/API boundaries
Frontend typing	TypeScript strict mode for new code
Frontend lint	ESLint; consistent formatter
Commit style	Conventional or otherwise machine-readable change categories
Branching	Short-lived feature branches; protected main branch
Reviews	At least one reviewer for normal change; additional security/domain review for high-risk modules
Generated files	API clients/schema artifacts generated reproducibly; avoid hand-editing generated code

23. Database Migration Policy

1. Every schema change is delivered through version-controlled Django migrations.
2. Destructive migrations use expand/migrate/contract patterns where data loss or downtime risk exists.
3. Data migrations are idempotent where feasible and MUST be reviewed for analytical-semantic changes.
4. Production migration commands are executed as an explicit release step with backup/rollback preparation.
5. Controlled vocabularies and typology catalogue versions are not silently rewritten by schema migrations.

24. Audit Implementation Standard

AuditEvent is a canonical append-oriented record. Application services emit audit events for material operations. Audit payloads SHOULD record object IDs and semantic changes without copying sensitive evidence bodies or protected-source identities into general logs.

Must audit	Examples
Identity/access	login/session events where needed, membership/role changes
Case governance	charter/scope changes, gates, ownership changes
Evidence	upload, integrity verification, derivative creation, restricted download
Entity analysis	merge, unmerge, status/confidence change
Reasoning	hypothesis status, assessment approval/version
Dissemination	product approval, export, recipient/share record
Administration	vocabulary, retention, policy and feature-flag changes

25. Observability and Logging

Operational telemetry SHALL help operate the platform without becoming a shadow copy of investigative data. Logs SHOULD use correlation IDs, route/action names, object IDs where safe, timings and error categories. Evidence text, secrets, tokens and protected-source identities SHALL NOT be included in routine logs.

- Metrics: request latency/error rate, queue depth, job failures, DB pool health, storage capacity, backup status.
- Health and metrics endpoints: GET /api/v1/health is part of the API contract (unauthenticated readiness probe; dependency status and latency only, no case content). The Prometheus /metrics endpoint is internal: scraped inside the deployment network, never routed by the reverse proxy, and not part of the API contract. [v0.1.3 · CR-17-08]
- Traces: optional OpenTelemetry traces with sensitive attribute allowlist.
- Application errors: structured logs with correlation IDs and user-safe error responses.
- Audit trail: separate canonical domain/audit record, not inferred from operational logs.

26. Backup, Restore and Disaster Recovery

Component	Backup requirement	Restore verification
PostgreSQL	Automated full/incremental or WAL-aware strategy appropriate to deployment	Periodic restore into isolated environment; integrity checks
Evidence object store	Versioned backup/replication with hash preservation	Sample object restore + hash comparison
Configuration	Infrastructure/config templates and controlled secrets recovery procedure	Recreate staging from documented baseline
Keycloak dependency	IdP realm/client configuration backup where organisation owns it	Test client/role recovery as applicable

RPO/RTO targets SHALL be set by the deployment owner in accordance with SRS availability requirements. A backup that has never been restored is not accepted as evidence of recoverability.

27. External Integration Boundary

P0 does not require OpenAleph, OpenSanctions, Flowintel, GraphSense, OpenSearch or a graph database. Integration adapters belong under integrations/ and SHALL map external data into ex-

licit candidate/reference objects with source/provider/time metadata. No connector may silently merge an external record into a canonical entity.

Integration	MVP	Phase 2+ approach
OpenAleph	Not required	Document/entity import with provenance and external IDs
OpenSanctions	Not required	Candidate screening result; analyst confirmation required
FollowTheMoney	Mapping not required	Versioned import/export mapping
Neo4j/Memgraph	Not required	Rebuildable graph projection
OpenSearch	Not required	Derived permission-aware search index
GraphSense	Not required	Attributed blockchain analytics connector
Flowintel	Not required	Workflow metadata interoperability only unless explicitly approved

28. Security Hardening Baseline

- TLS for all non-local traffic; session cookie `__Host-csaml_session` with `HttpOnly`, `Secure`, `SameSite=Lax`, `Path=/`; HSTS where deployment permits. *[v0.1.1 · A11]* Production settings SHALL force the `__Host-csaml_session` name and `Secure`; only an explicitly isolated local-development settings profile MAY override the cookie name and drop `Secure` for plain-http localhost (browsers refuse `__Host-*/Secure` cookies there). That profile SHALL NOT be usable in any shared or production environment. *[v0.1.2 · CR-I1-06]*
- CSRF protection (X-CSRFToken) for every unsafe method; API is same-origin and CORS is disabled by default. *[v0.1.1 · A11]*
- File upload size/type limits, quarantine path, safe content-disposition and no direct executable serving.
- Server-side object-level authorization for every canonical object access.
- Database user with least required privileges; separate operational/admin credentials.
- Rate limits for login-adjacent, search-heavy and export-heavy operations as appropriate.
- Security headers and frame/content controls suitable for analyst web application.
- No secrets in logs, frontend bundles, error pages or repository.

29. Performance and Capacity Baseline

MVP performance targets are designed for a small civil-society investigation team rather than hyperscale. Engineering SHOULD optimize predictable analyst interaction and evidence integrity before distributed scale.

Workload	Initial engineering target
Typical authenticated API read	p95 under 800 ms excluding large file transfer
Typical write command	p95 under 1.5 s excluding background processing
Case-scoped search	Interactive response for representative MVP corpus; target p95 under 2 s. Reconciled with SRS-NFR-PERF-002: both apply — global search first page ≤ 3 s and case-scoped search p95 < 2 s, measured on the synthetic reference corpus (backend/tests/synthetic_corpus.py, about 97,000 search rows; SRS-NFR-PERF-004); SHOULD-level targets, not guarantees. <i>[v0.1.3 · CR-I5-09]</i>
Graph view	Initial authorized subgraph loads interactively; pagination/expansion prevents unbounded graphs. Graph query budgets: depth ≤ 3 , 500 nodes, 1,500 edges, 4 s wall clock (partial result), 5 s statement timeout (503). <i>[v0.1.3 · CR-I5-02, CR-I5-06]</i>
Evidence upload	Streaming upload; request memory does not scale with file size
Background jobs	Observable progress/failure; retries bounded and idempotent

30. Accessibility and Analyst UX Engineering Rules

- Keyboard-accessible core workflows and visible focus states.
- Uncertainty classes SHALL not rely on color alone; labels/icons/text must survive monochrome export.
- Graph and timeline SHALL provide non-visual/tabular alternatives for critical facts.
- Forms SHOULD preserve unsaved-work protection for long analytical entries.
- Server validation errors SHALL map to understandable field/global messages without exposing internals.

31. Feature Flags

Feature flags MAY control incomplete or high-risk capabilities. Flags SHALL be evaluated server-side when they control authorization or sensitive processing. The MVP SHOULD reserve flags for external connectors, AI assist, advanced graph projection and experimental extraction—not for bypassing mandatory controls.

32. Architecture Decision Records

Material deviations from this specification SHALL be documented under docs/adr/ using sequential ADRs. An ADR records context, decision, alternatives, security/data implications, migration impact and supersession status.

```
docs/adr/
├── 0001-modular-monolith.md
├── 0002-postgresql-canonical-store.md
├── 0003-s3-evidence-storage.md      # S3 capability and evidence storage layout (product-neutral)
├── 0004-keycloak-oidc.md          # Keycloak OIDC with server-side BFF session
├── 0005-object-storage.md         # S3-compatible product selection
├── 0006-broker-cache-valkey.md    # Valkey 8.x pinned release and licence
└── ...
```

[v0.1.1 · A02, A03, A11] — ADR-0004 records the BFF session decision (confidential client, Authorization Code + PKCE, session cookie, CSRF). ADR-0005 records the chosen object-store product and release line, maintenance status, licence, patch path, S3 compatibility test evidence and backup/restore test evidence. ADR-0006 records the pinned Valkey release, its licence (BSD-3-Clause) and the matching SBOM entry.

[v0.1.1 · C19] ADR-0003 and ADR-0005 do not overlap: ADR-0003 records the S3-API capability and the evidence storage layout (bucket and object-key scheme, versioning, immutability/retention, encryption, original/derivative separation) and is product-neutral; ADR-0005 records only the selection of the object-store product and release line that implements that capability. A change of product is an ADR-0005 change and does not reopen ADR-0003.

33. Repository Documentation Set

File/path	Purpose
README.md	Developer quick start, architecture summary, local run commands
CONTRIBUTING.md	Branching, review, tests, commit and coding conventions
SECURITY.md	Security reporting, supported versions, handling of vulnerabilities
docs/architecture/	Logical/domain diagrams and runtime topology
docs/adr/	Architecture decisions
docs/api/	API usage and generated contract notes
docs/runbooks/	Deploy, rollback, backup/restore, incident procedures

File/path	Purpose
docs/data/	Data dictionary and migration semantics
fixtures/	Synthetic reproducible datasets only

34. MVP Module-to-Epic Mapping

Epic	Primary modules	Implementation focus
E0	common, audit, config, infra	Foundation, canonical envelope, storage, CI/CD
E1	iam, governance	OIDC, case membership, protected sources, policy
E2	cases	Case, charter, gates, tasks, activity
E3	sources, evidence	Provenance, original/derivative, hashes, extracts
E4	entities, relationships, assets	Resolution, merge/unmerge, ownership/control
E5	timeline, valueflows	Events, temporal view, value-flow model
E6	typologies, hypotheses, assessments	Structured reasoning, gaps and confidence
E7	search, graph	Permission-aware discovery and browser projection
E8	products, reviews, dissemination	Intelligence product, review, export/referral
E9	governance, common, infra	Vocabulary, retention, restore, hardening, release

35. Initial Delivery Sequence

Increment	Technical milestone	Exit evidence
I0	Repository + Compose + PostgreSQL + S3-compatible object store (ADR-0005) + Valkey + CI + audit skeleton [v0.1.1 · A02, A03]	Fresh checkout boots; migrations/tests pass; evidence round-trip works
I1	OIDC BFF session + policy + case workspace [v0.1.1 · A11]	Two users with different permissions demonstrate isolation
I2	Evidence + entity chain	Evidence-to-entity provenance trace demonstrated
I3	Relationships/assets/timeline/value flow	Case graph and flow preserve evidence and uncertainty class
I4	Typology/hypothesis/assessment	Competing hypotheses and confidence assessment complete
I5	Search + graph exploration	Permission-aware discovery with canonical back-links
I6	Product/review/dissemination	Independent review + approved export package
I7	Retention/backup/hardening/release	Full SRS end-to-end gate plus restore evidence

36. Technical Definition of Done

- Implementation traces to an SRS/engineering story ID.
- Domain logic resides in service/policy boundaries rather than incidental view code.
- Positive and negative authorization tests pass.
- Material state change emits required audit event.
- API schema and user-visible uncertainty/provenance semantics remain correct.
- Migration exists and has been exercised for schema changes.
- Unit/integration/E2E tests appropriate to risk pass in CI.
- No release-blocking security finding remains unowned/unaccepted.
- Operational/runbook impact is documented where applicable.

37. Technical Release Gate for MVP 0.1

MVP 0.1 SHALL NOT be considered technically releasable merely because all containers are healthy. The following system behavior must pass in one integrated environment:

6. Two analysts authenticate through OIDC and receive distinct role/case permissions.
7. A case is created, chartered and moved through required gate controls.
8. Original evidence is uploaded, hashed, preserved and cited through an EvidenceExtract.
9. Entities and relationships are created; a candidate merge is reviewed and remains reversible.
10. An asset, event timeline and mixed-class ValueFlow are built with evidence links.
11. Typology indicators and at least two competing hypotheses are recorded, including disconfirming evidence or gaps.
12. An assessment with confidence is created and independently reviewed.
13. An intelligence product is generated with evidence index and version metadata.
14. External dissemination is blocked before approval and logged after approval.
15. Unauthorized users cannot access the case through UI, API, search, graph or object download.
16. Backup/restore recovers canonical records and evidence while hashes remain valid.
17. A reviewer can reconstruct the material analytical chain from product back to sources.

38. Deferred Technical Decisions

Decision	Why deferred	Trigger
Dedicated OpenSearch cluster	PostgreSQL search sufficient for MVP	Representative corpus exceeds search/UX target
Neo4j/Memgraph	Canonical relational graph adequate initially	Path/network analytics become P1 priority
Kubernetes	Operational complexity not justified	Multi-node HA/managed platform requirements
Event bus	Celery/application events sufficient	Multiple independently deployed consumers emerge
Microservices	Domain boundaries still evolving	Scale/security/ownership evidence supports extraction
AI model gateway	AI is not P0	P1 AI assist approved with privacy/model controls
Phishing-resistant step-up (WebAuthn / passkeys), per-deployment step-up action list	TOTP step-up is the v0.1.4 baseline	v0.2 [v0.1.4 · CR-N-13]
Office-format (DOCX / XLSX) text extraction	No offline engine chosen	v0.2 [v0.1.4 · CR-N-09]

39. Conformance Checklist

Area	Minimum conformance evidence
Architecture	Modular monolith; canonical PostgreSQL; evidence object storage; derived projections rebuildable
Repository	Monorepo with backend/frontend/infra/docs/tests and documented ownership
Identity	OIDC with server-side BFF session plus local object/case authorization; protected-source compartment [v0.1.1 · A11]
Data	Stable IDs, provenance, uncertainty classes, reversible entity merges
Security	Server-side authorization, TLS, secret isolation, safe uploads, negative tests

Area	Minimum conformance evidence
Audit	Material actions append auditable canonical events
Quality	CI lint/type/test/security gates and migration checks
Operations	Containerized reproducible deploy; backups; tested restore; observability
Release	Integrated SRS/MVP acceptance path passes

40. Reference Repository Tree (Expanded)

```
cs-aml/  
├── backend/  
│   ├── config/  
│   ├── csaml/  
│   │   ├── common/  
│   │   ├── iam/  
│   │   ├── cases/  
│   │   ├── sources/  
│   │   ├── evidence/  
│   │   ├── entities/  
│   │   ├── relationships/  
│   │   ├── assets/  
│   │   ├── timeline/  
│   │   ├── valueflows/  
│   │   ├── typologies/  
│   │   ├── hypotheses/  
│   │   ├── assessments/  
│   │   ├── search/  
│   │   ├── graph/  
│   │   ├── products/  
│   │   ├── reviews/  
│   │   ├── dissemination/  
│   │   ├── governance/  
│   │   ├── audit/  
│   │   └── integrations/  
│   └── tests/  
├── frontend/  
│   ├── src/  
│   └── e2e/  
├── infra/  
│   ├── compose/  
│   ├── nginx/  
│   ├── monitoring/  
│   └── backup/  
├── docs/  
│   ├── architecture/  
│   ├── adr/  
│   ├── api/  
│   ├── data/  
│   └── runbooks/  
├── fixtures/  
├── scripts/  
├── tests/  
│   ├── integration/  
│   ├── security/  
│   └── release/  
└── .github/workflows/
```

41. Closing Engineering Decision

MVP technology posture

Build the distinctive CS-AML investigation and reasoning layer. Reuse mature infrastructure for identity, relational storage, object storage, async execution, telemetry and browser graph rendering. Avoid adding a dedicated search cluster, graph database, Kubernetes, microservices or AI gateway until a measured requirement justifies the operational cost.

This specification is the proposed technical baseline for implementation planning (draft for review). *[v0.1.1 · A01]* The next planning artifact **SHOULD** be the CS-AML Sprint & Milestone Plan, which assigns the engineering stories to ordered increments, identifies parallel work, defines review checkpoints, and establishes release evidence for each milestone.